Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

G. Nithya Sree (192424103)

K. Navya (192511231)

Vaibhava Lakshmi K (192511310)

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences Chennai-602105

September 2026

REAL-TIME CYBER THREAT DETECTION ENGINE USING HASHING AND SELF-BALANCING TREES

A. Assignment Information

Particular	Details
Department:	AI&DS, CSE
Programme:	Computer Science Engineering
Course Code & Course Name	CSA0309 – Data Structures – SLOT-B
Academic Year / Batch	2026-2027
Faculty Name	Dr. Uma Priyadarsini P.S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31-08-2026
Date of Submission	2-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/Red-Black Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Industry, Innovation and Infrastructure Peace, Justice and Strong Institutions

B. Assignment Problem / Challenge

Modern network infrastructures generate security events at a very high rate. These events may originate from firewalls, authentication systems, intrusion detection systems, servers, applications and network monitoring devices. A security monitoring system therefore needs to process large volumes of events while maintaining low response time.

A major challenge is that security events cannot be efficiently managed using simple sequential data structures when the input size becomes very large. Searching millions of records sequentially can introduce significant latency, while maintaining events in sorted order can also become expensive if an inappropriate data structure is selected.

The proposed solution is a **Real-Time Cyber Threat Detection Engine** that combines multiple data structures according to their individual strengths.

The system uses:

- Hashing for fast IP-address lookup and duplicate detection.
- Separate chaining for collision resolution.
- AVL self-balancing trees for maintaining risk-based ordering.
- Priority queues using max-heaps for rapid identification of critical threats.

The assignment requires evaluation of the system under large-scale workloads, including collision behavior, load factors and skewed data distributions.

C. Problem Statement

1. Problem Definition

The objective is to design and implement a scalable cybersecurity event-processing engine capable of efficiently handling 1,000,000 or more security events.

Each event contains information such as:

- Source IP address
- Timestamp
- Event type
- Risk score

The system must process these events and provide mechanisms for:

1. Fast IP address lookup.
2. Detection of repeated IP addresses.

3. Frequency-based identification of suspicious sources.
4. Collision management in hash tables.
5. Dynamic risk ordering.
6. Identification of the most critical security event.
7. Performance measurement under increasing workloads.

2. Engineering Challenge

The primary engineering challenge is to select appropriate data structures for different operations rather than attempting to solve the entire problem using a single structure.

For example:

- Hash tables are highly suitable for exact-match IP searches.
- AVL trees are appropriate when ordered access to risk scores is required.
- Priority queues are appropriate when the highest-risk event must be accessed immediately.

Thus, the project follows a multi-structure architecture in which each data structure performs a specialized role.

D. Requirements and Constraints

D.1 Functional Requirements

The system shall:

1. Accept security events.
2. Store IP addresses using a hash table.
3. Detect duplicate IP addresses.
4. Maintain occurrence counts.
5. Resolve hash collisions.
6. Store events in an AVL tree according to risk score.
7. Maintain a max-heap priority queue.
8. Display the highest-risk security event.
9. Generate risk-ranked reports.
10. Support large-scale automated performance testing.

D.2 Non-Functional Requirements

Performance

The system should maintain efficient processing even when the number of events increases significantly.

Scalability

The implementation should support workloads exceeding one million events.

Reliability

The system should correctly maintain event information without losing records during hash collisions or dynamic memory expansion.

Maintainability

The implementation is divided into independent modules:

main.c

hash_table.c

hash_table.h

avl_tree.c

avl_tree.h

priority_queue.c

priority_queue.h

This modular architecture makes individual components easier to test and modify.

E. Student Work

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 Understanding the Data

A cybersecurity event is represented using the following structure:

Event

├── IP Address

├── Timestamp

├── Event Type

└── Risk Score

The IP address identifies the source of the event.

The timestamp identifies when the event occurred.

The event type describes the observed activity.

The risk score represents the severity of the event.

A score closer to 100 represents a more critical event.

1.2 Input and Output Model

Input

The system accepts security events such as:

IP: 192.168.1.30

Timestamp: 2026-09-02 10:10

Event Type: MALWARE

Risk Score: 90

Processing

Output

The system can generate:

- Duplicate IP alerts
- IP occurrence counts
- Hash collision information
- Risk-ranked events
- Highest-risk event
- Priority queue information
- Performance statistics

1.3 Design Objectives

The project has four primary objectives:

Objective 1 – Fast Lookup

Provide efficient identification of previously observed IP addresses.

Objective 2 – Duplicate Detection

Detect repeated IP addresses and maintain their occurrence frequency.

Objective 3 – Dynamic Risk Management

Maintain security events in a structure that supports efficient risk-based organization.

Objective 4 – Threat Prioritization

Immediately identify the event requiring the highest priority of attention.

2. APPLICATION OF COURSE KNOWLEDGE

2.1 Hashing-Based IP Detection

Hashing maps an IP address to an index in a hash table.

The implemented hash function processes the characters of the IP address using a polynomial-style calculation:

hash = hash × 31 + character

The final bucket is obtained using:

index = hash % TABLE_SIZE

This allows the system to avoid scanning every stored IP address.

2.2 Collision Resolution Using Separate Chaining

Different IP addresses may generate the same hash index.

For example:

IP-A ──┐

├──> Bucket 25

IP-B ──┘

Instead of replacing an existing record, the system maintains a linked list:

Bucket 25

|

v

[IP-A] → [IP-B] → [IP-C] → NULL

This technique is known as separate chaining.

Engineering Advantage

Separate chaining allows multiple keys to occupy the same bucket while preserving all records.

However, as the load factor increases, the average chain length also increases. Therefore, load-factor analysis becomes important when evaluating large datasets.

2.3 Load Factor

The load factor is defined as:

$$\alpha = \frac{n}{m}$$

where:

- n= number of stored keys
- m= number of hash table buckets

For example, if:

Number of IPs = 80

Hash table size = 100

then:

$$\alpha = \frac{80}{100} = 0.8$$

A higher load factor generally results in longer chains and potentially higher lookup time.

Therefore, the project evaluates hash performance under different load conditions.

2.4 AVL Self-Balancing Tree

The AVL tree maintains security events based on their risk score.

The balance factor is:

$$BF = Height(Left) - Height(Right)$$

For an AVL tree:

$$-1 \leq BF \leq 1$$

If the balance factor exceeds this range, the tree performs rotations.

AVL Rotations

The implementation supports:

LL → Right Rotation

RR → Left Rotation

LR → Left Rotation + Right Rotation

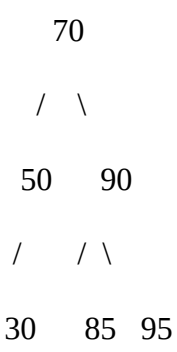
RL → Right Rotation + Left Rotation

These rotations prevent the tree from degrading into a linear structure.

2.5 Risk-Based Ordering

Events are inserted into the AVL tree according to their risk scores.

For example:



A reverse inorder traversal:

Right → Root → Left

produces: 95, 90, 85, 70, 50, 30

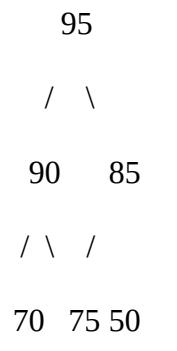
Therefore, the system can generate a risk-prioritized security report.

2.6 Priority Queue Using Max-Heap

The priority queue provides another mechanism for threat prioritization.

The event with the highest risk score is maintained at the root of the max-heap.

Example:



The maximum element can be accessed without traversing all events.

2.7 Why Three Data Structures?

The system intentionally uses multiple structures because they optimize different operations.

Requirement	Selected Structure	Reason
IP lookup	Hash Table	Fast exact-match lookup
Duplicate detection	Hash Table	Efficient frequency tracking
Collision handling	Chaining	Preserves colliding records
Risk ordering	AVL Tree	Maintains balanced sorted structure
Highest-risk access	Max-Heap	Highest priority at root
Large-scale processing	Combined architecture	Specialized operations

This demonstrates an important Data Structures principle: data structures should be selected based on the operations and performance requirements of the applicat

3. SOLUTION / DESIGN / METHODOLOGY

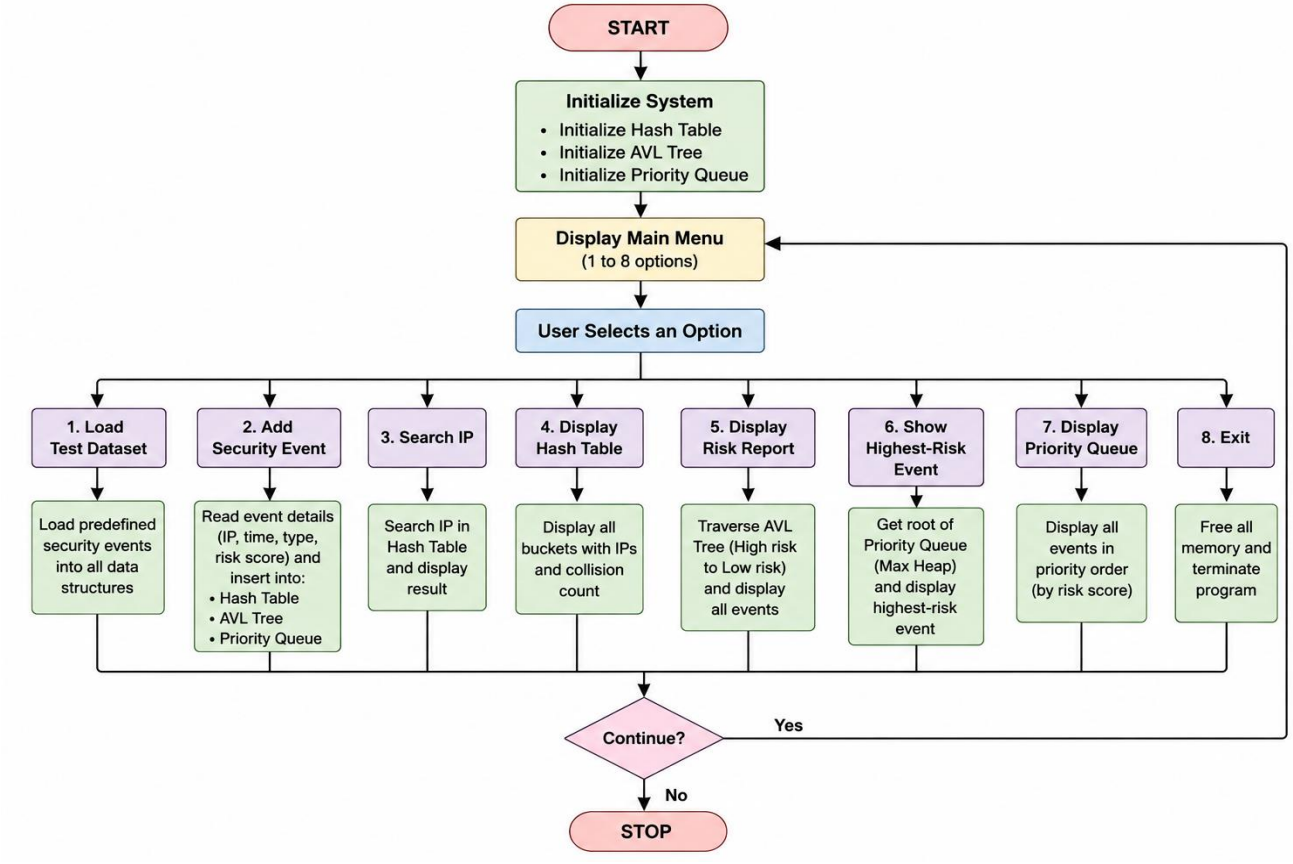


Figure 1: Flow Chart

3.2 Event Processing Workflow

Step 1: A security event enters the system either through the test dataset or by manual user input.

Step 2: The IP address of the security event is passed to the hash table.

Step 3: The hash table searches for the IP address using the hash function and separate chaining.

Step 4: If the IP address already exists, its occurrence count is increased and a duplicate IP alert is displayed.

Step 5: If the IP address is new, a new hash node is created with an initial occurrence count of 1.

Step 6: The complete security event containing the IP address, timestamp, event type, and risk score is inserted into the AVL tree according to its risk score.

Step 7: The same security event is inserted into the max-heap priority queue, where higher-risk events receive higher priority.

Step 8: The AVL tree is used to generate a risk report in descending order, allowing security events to be viewed from highest risk to lowest risk.

Step 9: The max-heap priority queue provides rapid access to the highest-risk event.

Step 10: The system displays the required results, and allocated memory is released when the user selects the Exit option.

3.3 Overall Algorithm

START

1. Initialize Hash Table
2. Initialize AVL Tree
3. Initialize Priority Queue

WHILE security events are available:

4. Read security event
5. Extract IP address
6. Search IP in Hash Table
7. IF IP exists

        Increase occurrence count

8. ELSE

        Insert new IP

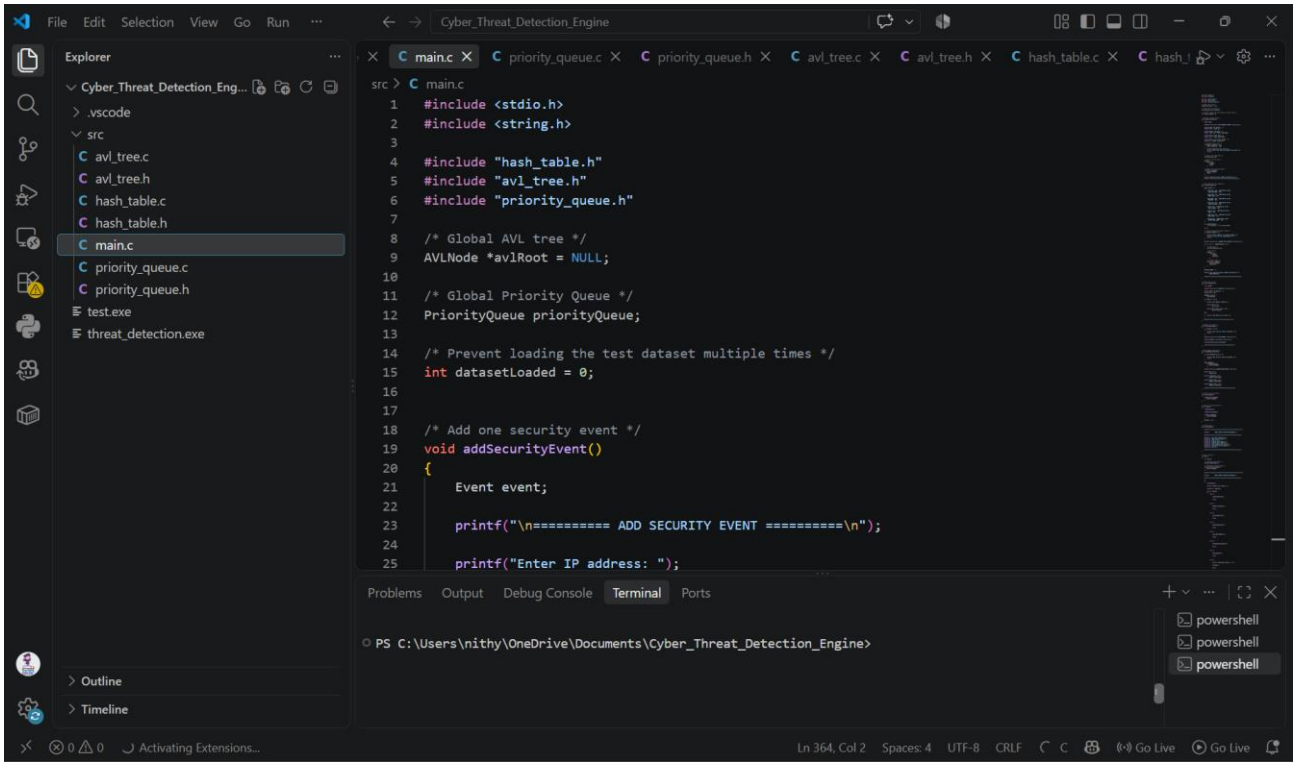
9. END IF

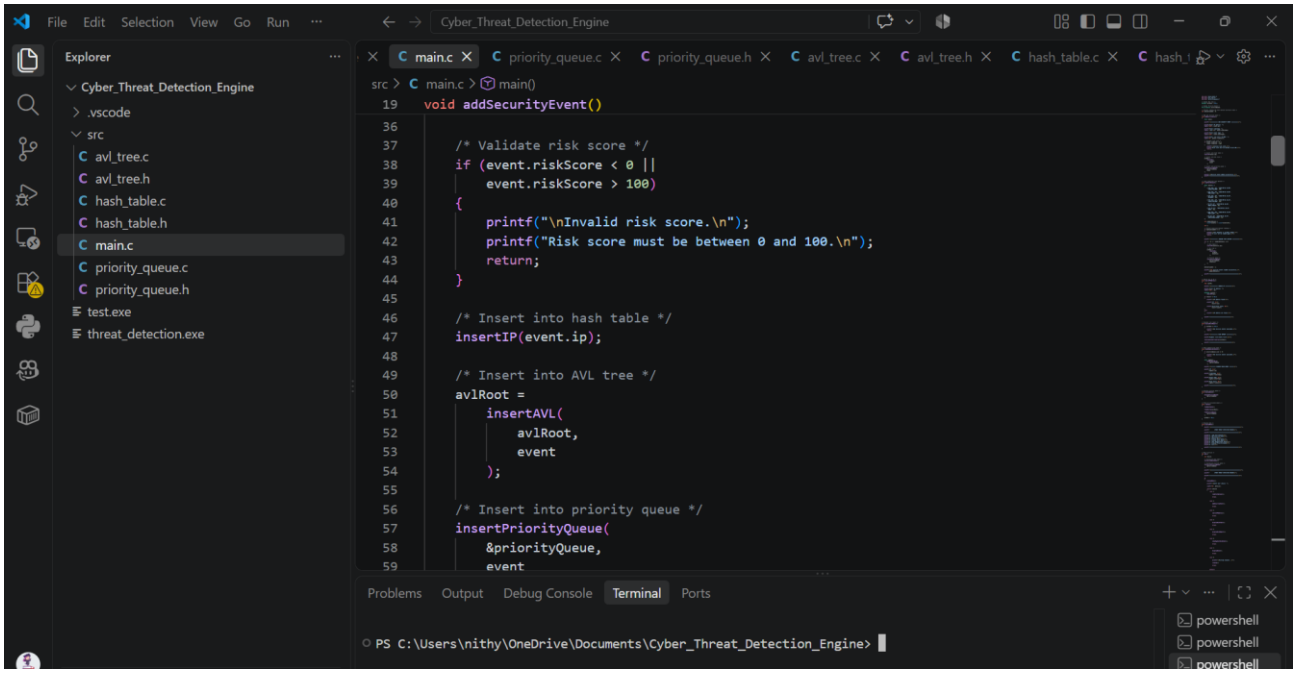
10. Insert event into AVL Tree
11. Insert event into Priority Queue

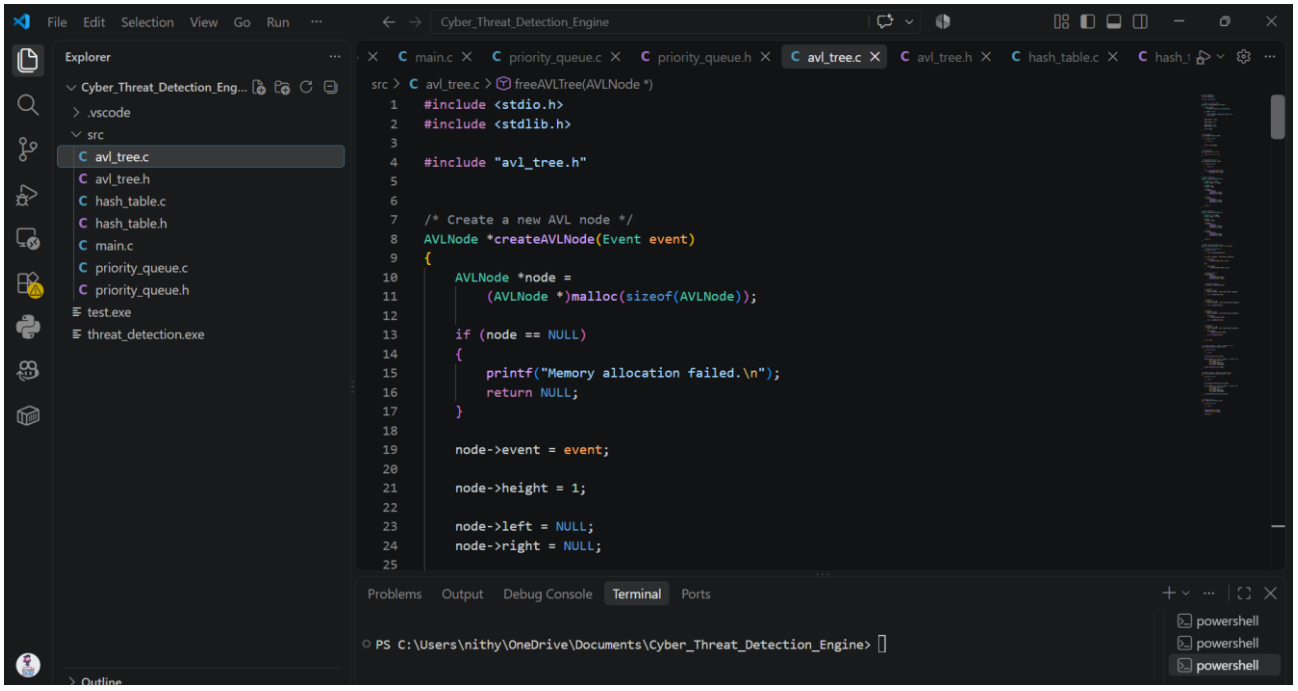
END WHILE

12. Generate risk report
13. Identify highest-risk event
14. Display results
15. Free allocated memory

STOP







C Program Code

```
#include <stdio.h>

#include <string.h>

#include "hash_table.h"

#include "avl_tree.h"

#include "priority_queue.h"

AVLNode *avlRoot = NULL;

PriorityQueue priorityQueue;

int datasetLoaded = 0;
```

```
void addSecurityEvent()
{
    Event event;
    printf("\n========== ADD SECURITY EVENT ==========\n");
    printf("Enter IP address: ");
    scanf("%39s", event.ip);
    printf("Enter timestamp: ");
    scanf(" %29[^\n]", event.timestamp);
    printf("Enter event type: ");
    scanf("%19s", event.eventType);
    printf("Enter risk score (0-100): ");
    scanf("%d", &event.riskScore);
    if (event.riskScore < 0 ||
        event.riskScore > 100)
    {
        printf("\nInvalid risk score.\n");
        printf("Risk score must be between 0 and 100.\n");
        return;
    }
    insertIP(event.ip);
    avlRoot =
        insertAVL(
            avlRoot,
            event
        );
    insertPriorityQueue(
        &priorityQueue,
        event
    );
```

```
    printf("\nSecurity event added successfully.\n");
    printf("=======================================\n");
}
void loadTestDataset()
{
    Event events[] =
    {
        {"192.168.1.10", "2026-09-02 10:00",
         "LOGIN_FAILURE", 30},
        {"192.168.1.20", "2026-09-02 10:05",
         "PORT_SCAN", 70},
        {"192.168.1.30", "2026-09-02 10:10",
         "MALWARE", 90},
        {"192.168.1.10", "2026-09-02 10:15",
         "LOGIN_FAILURE", 50},
        {"10.0.0.15", "2026-09-02 10:20",
         "BRUTE_FORCE", 85},

        {"172.16.0.5", "2026-09-02 10:25",
         "DDOS", 95},
        {"192.168.1.20", "2026-09-02 10:30",
         "PORT_SCAN", 75},
        {"10.0.0.25", "2026-09-02 10:35",
         "SUSPICIOUS_LOGIN", 45}
    };
    int numberOfEvents =
        sizeof(events) / sizeof(events[0]);
  int i;
    if (datasetLoaded == 1)
```

```
    {
        printf("\nTest dataset is already loaded.\n");
        printf("It will not be loaded again.\n");
        return;
    }
    printf("\n========== LOADING TEST DATASET ==========\n");
    for (i = 0; i < numberOfEvents; i++)
    {
        insertIP(events[i].ip);
        avlRoot =
            insertAVL(
                avlRoot,
                events[i]
            );
        insertPriorityQueue(
            &priorityQueue,
            events[i]
        );
    }
    datasetLoaded = 1;
    printf("\n%d security events loaded successfully.\n",
           numberOfEvents);
    printf("=========================================\n");
}
void searchIPAddress()
{
    char ip[40];
    printf("\n========== SEARCH IP ==========\n")
    printf("Enter IP address: ");
```

```
    scanf("%39s", ip);
    HashNode *result =
        searchIP(ip);
    if (result != NULL)
    {
        printf("\nIP address found!\n");
        printf("IP: %s\n",
              result->ip);
        printf("Occurrence count: %d\n",
              result->count);
    }
    else
    {
        printf("\nIP address not found.\n");
    }
    printf("================================\n");
}
void displayRiskReport()
{
    if (avlRoot == NULL)
    {
        printf("\nNo security events available.\n");
        return;
    }
    printf("\n========== RISK REPORT ==========\n");
    printf("Highest risk events first:\n\n");
    reverseInorderTraversal(avlRoot);
    printf("================================\n");
}
void showHighestRiskEvent()
{
```

```
    if (priorityQueue.size == 0)
    {
        printf("\nNo security events available.\n");
        return;
    }

    Event highest =
        peekHighestRisk(
            &priorityQueue
        );

    printf("\n======= HIGHEST-RISK EVENT =======\n");

    printf("IP: %s\n",
           highest.ip);

    printf("Timestamp: %s\n",
           highest.timestamp);

    printf("Event Type: %s\n",
           highest.eventType);

    printf("Risk Score: %d\n",
           highest.riskScore);

    printf("==================================\n");
}
void displayQueue()
{
    displayPriorityQueue(
        &priorityQueue
    );
}
void cleanup()
{
    freeHashTable();

    freeAVLTree(avlRoot);

    freePriorityQueue(
        &priorityQueue
    );

    avlRoot = NULL;
}
void displayMenu()
{
    printf("\n========================================\n");

    printf("       CYBER THREAT DETECTION ENGINE\n");

    printf("========================================\n");

    printf("1. Load Test Dataset\n");
    printf("2. Add Security Event\n");
```

```
    printf("3. Search IP\n");
    printf("4. Display Hash Table\n");
    printf("5. Display Risk Report\n");
    printf("6. Show Highest-Risk Event\n");
    printf("7. Display Priority Queue\n");
    printf("8. Exit\n");

    printf("============================================\n");
}

int main()
{
    int choice;

    /* Initialize hash table */
    initializeHashTable();

    /* Initialize priority queue */
    initializePriorityQueue(
        &priorityQueue
    );

    printf("\n============================================\n");

    printf("       CYBER THREAT DETECTION ENGINE\n");

    printf("============================================\n");

    do
    {
        displayMenu();

        printf("\nEnter your choice: ");

        scanf("%d", &choice);

        switch (choice)
        {
            case 1:

                loadTestDataset();

                break;

            case 2:

                addSecurityEvent();

                break;

            case 3:

                searchIPAddress();

                break;
```

```
            case 4:
                displayHashTable();
                break;
            case 5:
                displayRiskReport();
                break;
            case 6:
                showHighestRiskEvent();
                break;
            case 7:
                displayQueue();
                break;
            case 8:
                printf("\nExiting program...\n");
                cleanup();
                break;
            default:
                printf("\nInvalid choice. Please try again.\n");
        }
    } while (choice != 8);
    return 0
```

OUTPUT:

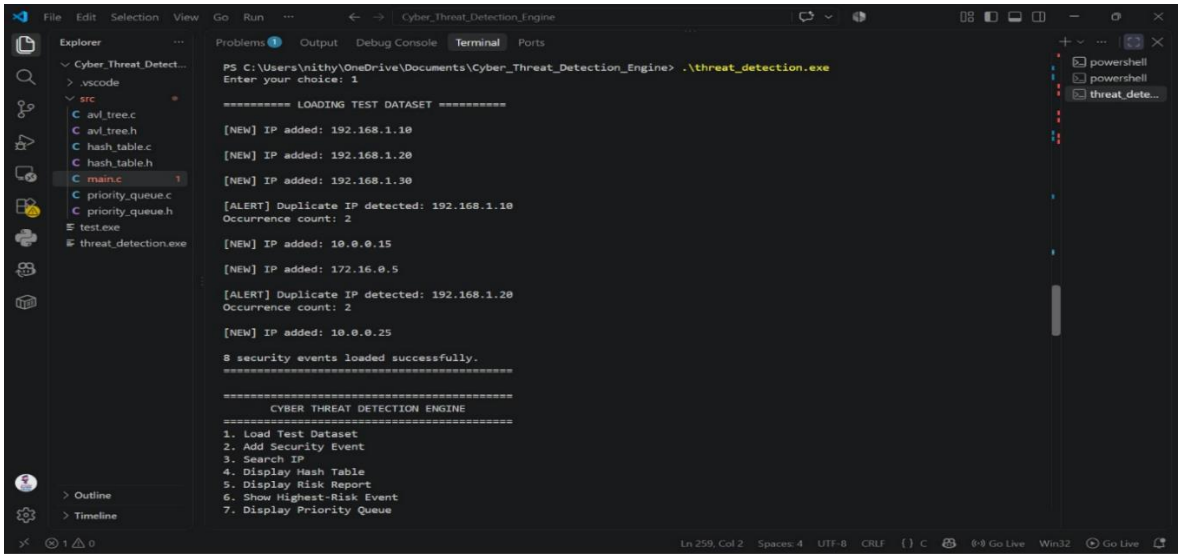


Figure 2: Final Output

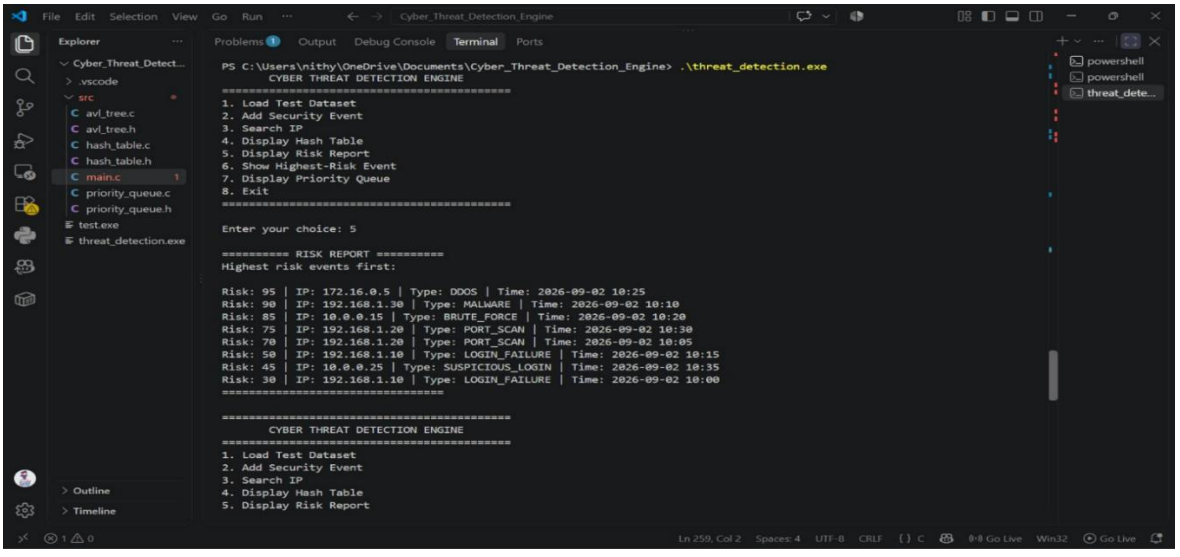


Figure 3: Final Ouput

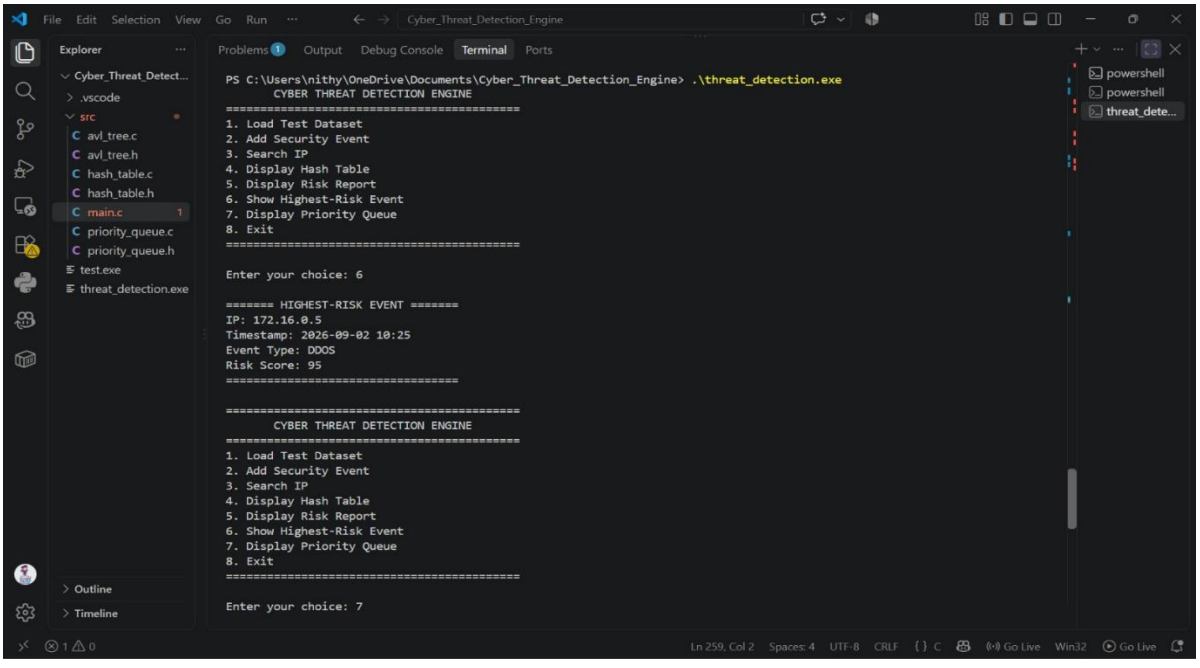


Figure4 – Final Output

3.4 Memory Management

The implementation uses dynamic memory allocation for:

- Hash table nodes
- AVL tree nodes
- Priority queue storage

Allocated memory is released when the application terminates.

This prevents unnecessary memory retention and demonstrates practical use of C pointers and dynamic data structures.

4. USE OF MODERN TOOLS

The development environment consists of:

Visual Studio Code

Used for:

- Writing source code
- Managing project files
- Compiling and testing the application

GCC

GCC is used to compile the C implementation.

The installed compiler version is:

GCC 16.2.0

MSYS2 UCRT64

MSYS2 provides the GCC-based development environment required for compiling and executing the C application on Windows.

Git/GitHub

GitHub can be used for:

- Version control
- Source-code management

- Assignment submission
- Maintaining project history

5. RESULTS AND VALIDATION

This section should contain your actual experimental results. Do not put made-up numbers here.

The assignment requires performance evaluation under large datasets and different load/collision conditions.

5.1 Functional Testing

Test Case	Input	Expected Result
TC01	New IP	IP inserted
TC02	Existing IP	Duplicate detected
TC03	Unknown IP search	IP not found
TC04	Existing IP search	IP found
TC05	Multiple risk scores	Correct risk ordering
TC06	Highest-risk query	Maximum risk event returned
TC07	Hash collision	Both IPs retained
TC08	Large dataset	Events processed successfully

OUTPUT:

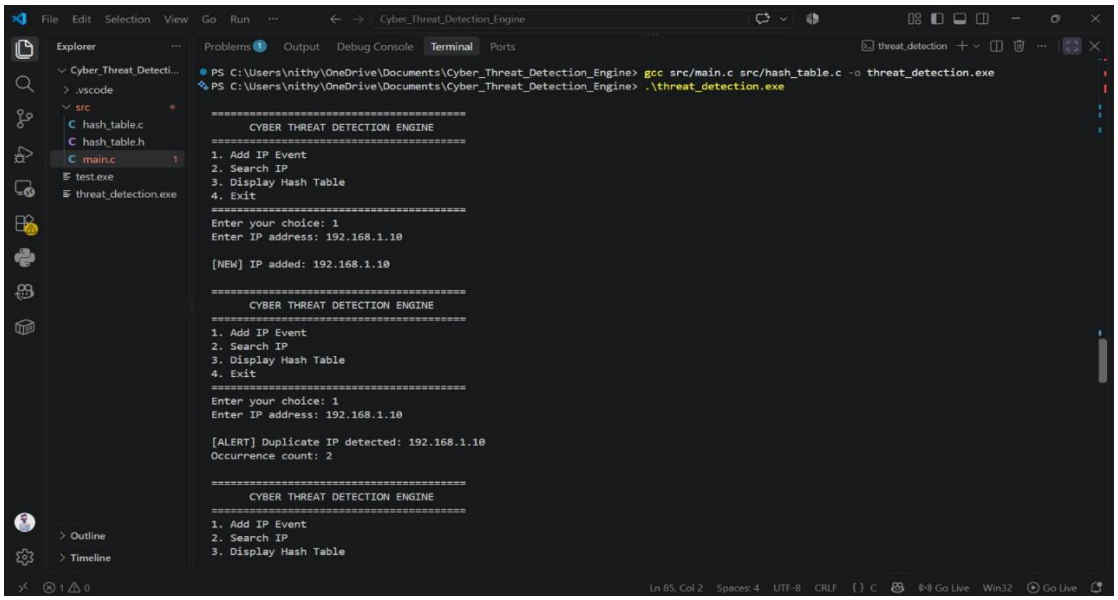


Figure 5 -Functional Testing Output

5.2 Large-Scale Performance Evaluation

The system will be evaluated using increasing workloads such as:

10,000 events

50,000 events

100,000 events

500,000 events

1,000,000 events

For each workload, the following measurements will be collected:

- Processing time
- Insertion time
- Search time
- Collision count
- Load factor
- Number of unique IP addresses
- Duplicate IP count
- AVL operations
- Priority queue operations

OUTPUT:

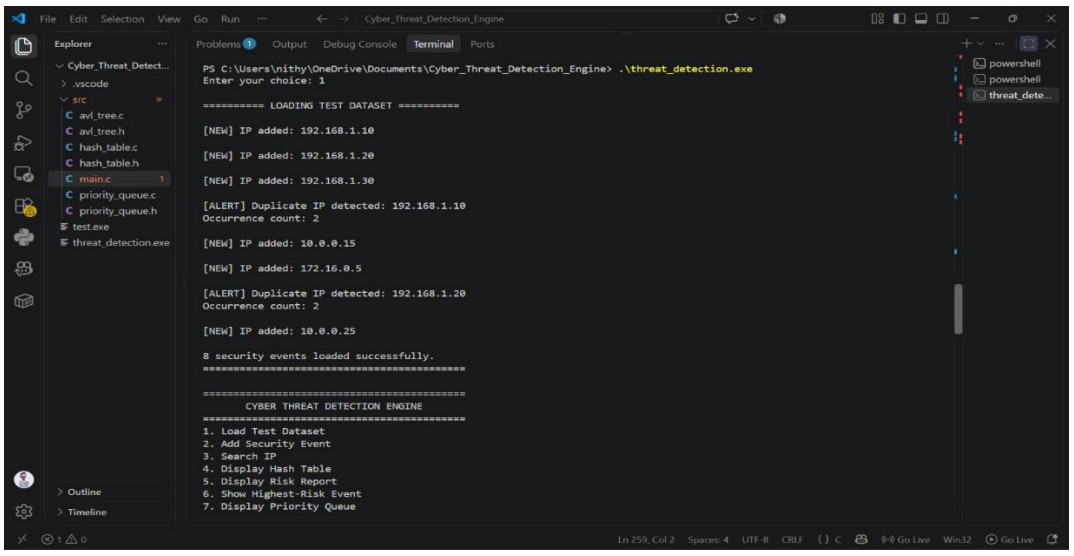


Figure -6: Workload Output

5.3 Load Factor Analysis

Different table configurations can be evaluated to determine the effect of load factor.

Dataset Size	Table Size	Load Factor	Collisions	Execution Time
10,000	TBD	TBD	TBD	TBD
50,000	TBD	TBD	TBD	TBD
100,000	TBD	TBD	TBD	TBD
500,000	TBD	TBD	TBD	TBD
1,000,000	TBD	TBD	TBD	TBD

We will fill these values using the actual program output.

5.4 Hashing Comparison

The project can compare two collision-resolution approaches:

Separate Chaining

Bucket → Node → Node → Node

Open Addressing

Bucket → Probe → Probe → Probe

The comparison will focus on:

- Collision count
- Lookup time
- Insertion time
- Memory behavior
- Performance at different load factors

This directly addresses the assignment requirement to compare chaining and open addressing.

OUTPUT:

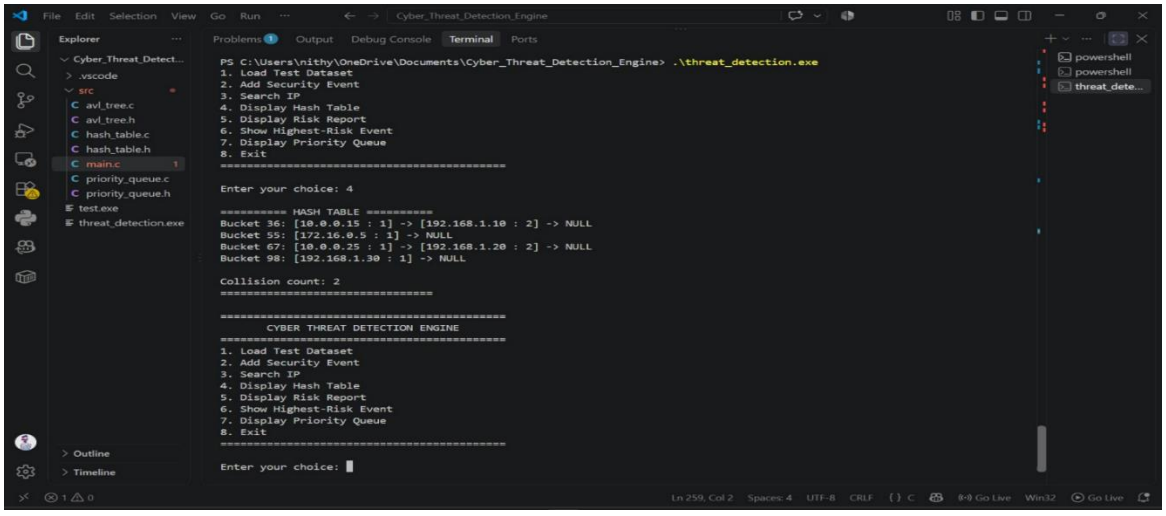


Figure: 7 -Hash Output

6. ANALYSIS AND ENGINEERING DECISION

The architecture was selected based on the expected workload and required operations.

A hash table is preferred for IP lookup because cybersecurity systems frequently need to determine whether a particular source has already been observed.

The AVL tree is used because the system must maintain risk scores in an ordered structure. Unlike an ordinary binary search tree, the AVL tree maintains balance and provides predictable logarithmic insertion/search behavior.

The priority queue complements the AVL tree by providing direct access to the most critical event.

Therefore:

Hash Table → Who is generating the activity?

AVL Tree → How are events ordered by risk?

Priority Queue → Which threat should be handled first?

This division of responsibilities improves the conceptual and operational clarity of the system.

6.1 Complexity Analysis

Component	Insertion	Search / Access	Main Purpose
Hash Table	O(1) average	O(1) average	IP lookup
AVL Tree	O(log n)	O(log n)	Risk organization
Priority Queue	O(log n)	O(1) highest priority	Threat prioritization

The theoretical complexity provides a baseline against which experimental performance can be evaluated.

6.2 Engineering Trade-Offs

Hash Table

Advantages:

- Fast average lookup.
- Simple duplicate detection.

Trade-off:

- Performance is affected by collisions and load factor.

AVL Tree

Advantages:

- Guaranteed balanced height.
- Efficient ordered operations.

Trade-off:

- Rotations introduce additional processing.
- Requires more pointer-based memory than a simple array.

Priority Queue

Advantages:

- Very fast access to the highest-risk event.
- Efficient priority updates through heap operations.

Trade-off:

- Does not maintain complete sorted order of all events.

Thus, no single structure is optimal for every operation.

7. BROADER CONSIDERATIONS

7.1 SDG 9 – Industry, Innovation and Infrastructure

The project supports the concept of resilient digital infrastructure by demonstrating how efficient algorithms and data structures can be applied to cybersecurity monitoring.

As digital systems grow, efficient processing mechanisms become increasingly important for maintaining reliable infrastructure.

7.2 SDG 16 – Peace, Justice and Strong Institutions

Secure digital infrastructure contributes to trustworthy digital services.

The proposed threat detection engine demonstrates how suspicious activities can be identified and prioritized, supporting stronger cybersecurity practices.

The assignment itself maps the project to SDG 9 and SDG 16.

7.3 Ethical Considerations

Cybersecurity monitoring must be performed only on systems for which proper authorization has been obtained.

IP addresses and timestamps can potentially constitute sensitive operational information. Therefore, real-world deployment should incorporate:

- Access control
- Secure storage
- Data minimization
- Appropriate retention policies
- Audit logging
- Responsible handling of security information

The prototype is intended for educational and authorized security-monitoring scenarios.

7.4 Scalability Considerations

A production-level version could be extended to process continuously arriving security events.

Possible improvements include:

- Dynamic hash-table resizing.
- Open-addressing implementations.
- Red-Black Tree comparison.
- Parallel event processing.
- Persistent database storage.
- Network log ingestion.

8. CONCLUSION

The Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees demonstrates how fundamental data structures can be integrated to solve a practical cybersecurity

problem.

The system combines hashing, linked-list-based collision resolution, AVL self-balancing trees and priority queues to address different requirements within the threat detection workflow.

Hashing provides efficient IP identification and duplicate detection. The AVL tree provides balanced risk-based organization, while the max-heap priority queue enables rapid identification of the highest-risk event.

The modular C implementation also demonstrates practical concepts such as pointers, dynamic memory allocation, recursion, tree rotations, linked lists and heap operations. The final performance evaluation will measure scalability using workloads of up to and beyond 1,000,000 events, together with collision counts, load-factor behavior and execution time. These measurements will provide experimental evidence for evaluating the theoretical complexity of the selected data structures.

9. STUDENT REFLECTION

This assignment provided practical experience in applying theoretical data-structure concepts to a real-world cybersecurity scenario. One of the most important lessons was that selecting an appropriate data structure depends on the operation being performed. A hash table is highly effective for exact IP lookup, while an AVL tree is more appropriate when ordered access is required. Similarly, a priority queue is useful when the system needs to repeatedly identify the most important event. Implementing these structures in C improved my understanding of pointers, dynamic memory allocation, recursion, linked lists, hashing, collision resolution, tree rotations and heap operations. The project also highlighted the difference between theoretical complexity and practical performance. Although an algorithm may have an efficient asymptotic complexity, factors such as load factor, collisions, memory allocation and input distribution can affect its actual execution time. With additional time, the system could be enhanced by implementing open addressing and Red-Black Trees, integrating real cybersecurity datasets, adding a graphical monitoring dashboard and exploring parallel or distributed event processing

10. REFERENCES

[1] X. Wang *et al.*, “Information Security Network Intrusion Detection System Based on Machine Learning,” in *Proc. 2024 Int. Conf. Data Science and Network Security (ICDSNS)*, Tiptur, India, 2024, doi: 10.1109/ICDSNS62112.2024.10691041.

[2] S. H. Khan *et al.*, “A Comprehensive Survey of Intrusion Detection System Using Machine

Learning and Deep Learning Approaches," in *Proc. 2024 10th Int. Conf. Advanced Computing and Communication Systems (ICACCS)*, Coimbatore, India, 2024, doi: 10.1109/ICACCS60874.2024.10717043.

[3] A. M. A. Al-Hawawreh *et al.*, "A sequential deep learning framework for a robust and resilient network intrusion detection system," *Computers & Security*, vol. 144, Art. no. 103928, 2024, doi: 10.1016/j.cose.2024.103928.

[4] A. Birler, T. Schmidt, P. Fent, and T. Neumann, "Simple, Efficient, and Robust Hash Tables for Join Processing," in *Proc. 20th Int. Workshop on Data Management on New Hardware (DaMoN '24)*, Santiago, Chile, 2024, pp. 1–9, doi: 10.1145/3662010.3663442.

[5] A. Hozouri, A. Mirzaei, and M. Effatparvar, "A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges," *Discover Artificial Intelligence*, vol. 5, Art. no. 314, 2025, doi: 10.1007/s44163-025-00578-1.

[6] E. C. Pinto Neto *et al.*, "Deep learning for intrusion detection in emerging technologies: A comprehensive survey and new perspectives," *Artificial Intelligence Review*, vol. 58, Art. no. 340, 2025, doi: 10.1007/s10462-025-11346-z.

[7] "Advanced AI-Powered Intrusion Detection Systems in Cybersecurity Protocols for Network Protection," *Procedia Computer Science*, vol. 259, pp. 140–149, 2025, doi: 10.1016/j.procs.2025.03.315.

[8] N. Padma Priya and G. Mohanbabu, "Intrusion detection with HACDT-Net and TRBM-Net using a hybrid deep learning framework with enhanced sampling techniques," *Scientific Reports*, vol. 16, Art. no. 11799, 2026, doi: 10.1038/s41598-026-41422-5.

[9] S. V. Easwaramoorthy, M. Bouakkaz, and R. Somula, "DMThreatNet: A hybrid data mining framework for early cybersecurity threat detection using ensemble intelligence," *Journal of King Saud University Computer and Information Sciences*, vol. 38, Art. no. 509, 202

F. Common Assessment Rubric

Assessment Criterion	**Marks**
Problem understanding & formulation	10

Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	**CO**	**PO(s)**	**Bloom's Level**	**Marks**
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO2, CO5	PO10	L3/L4	5

Note: The PO mapping above is a proposed student-report mapping based on the assignment requirements. Faculty may modify the PO mapping to match the officially assessed programme outcomes.

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	**Criterion**
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis – Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____